ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 2ND YEAR

DATABASES

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 2nd year Databases paper 1 · practice a.

Written by Academic Central for practice, following second-year Computer Science syllabuses at South African universities (database systems). It is not a university memorandum.

How to mark: each ✓ is one mark. Accept any correct SQL that gives the required result (other column orders, aliases, JOIN … ON or WHERE joins). Accept other sensible ER designs that model the scenario correctly. Deduct at most one mark per query for small syntax slips.

QUESTION 1

1.1 A ✓
1.2 B ✓
1.3 B ✓
1.4 A ✓
1.5 B ✓
1.6 B ✓
1.7 B ✓
1.8 A ✓
1.9 B ✓
1.10 C ✓

(10)

[10]

QUESTION 2

2.1 Software that creates, stores, manages and controls access to a database ✓. Any TWO: less duplicated data ✓; data stays consistent; several users can share the data at the same time ✓; security and access control; backup and recovery; data is independent of the programs (3)

2.2 Entity integrity: every table has a primary key, and no part of it may be null ✓✓
Referential integrity: every foreign key value must match an existing primary key value in the table it refers to (or be null) ✓✓ (4)

2.3 Candidate key: any minimal set of columns that uniquely identifies a row ✓
Primary key: the candidate key chosen to identify rows ✓
Composite key: a key made of two or more columns ✓ (3)

2.4 A transaction is a group of operations that must be done completely or not at all (one logical unit of work) ✓
COMMIT makes all the changes of the transaction permanent ✓
ROLLBACK undoes all the changes since the transaction started ✓ (3)

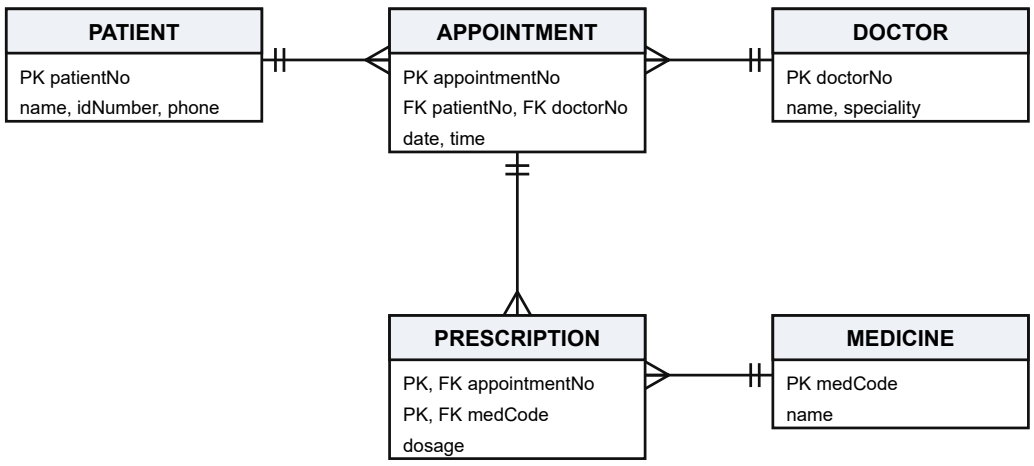
2.5 A data structure (e.g. a B-tree) that makes searching on a column faster ✓. Disadvantage, any ONE: inserts, updates and deletes become slower; uses extra storage ✓ (2)

[15]

QUESTION 3

3.1 Patient (patientNo) ✓ Doctor (doctorNo) ✓ Appointment (appointmentNo) ✓ Medicine (medCode) ✓ (4)

3.2



Four entities with keys ✓; Patient 1 to many Appointment ✓✓; Doctor 1 to many Appointment ✓✓; many-to-many between Appointment and Medicine resolved with Prescription ✓; both 1-to-many links to Prescription ✓; dosage in Prescription ✓ (8)

3.3 Patient(patientNo, name, idNumber, phone) ✓ (6)
Doctor(doctorNo, name, speciality) ✓
Appointment(appointmentNo, date, time, patientNo FK, doctorNo FK) ✓✓
Medicine(medCode, name) ✓
Prescription(appointmentNo FK, medCode FK, dosage) ✓

3.4 In Prescription ✓, because the dosage depends on BOTH the appointment and the medicine (the same medicine has different dosages at different appointments) ✓ (2)

[20]

QUESTION 4

4.1 Insertion: a new product cannot be stored until someone orders it ✓ (3)
Update: if N. Dube's name changes, it must be changed in several rows, or the data becomes inconsistent ✓
Deletion: deleting order 502 also deletes the only record of customer S. Pillay ✓

4.2 OrderNo → OrderDate, CustomerNo ✓ (4)
CustomerNo → CustomerName ✓
ProductCode → ProductName, UnitPrice ✓
OrderNo, ProductCode → Qty ✓

4.3 Customer(CustomerNo, CustomerName) ✓✓ (8)
Orders(OrderNo, OrderDate, CustomerNo FK) ✓✓
Product(ProductCode, ProductName, UnitPrice) ✓✓
OrderLine(OrderNo FK, ProductCode FK, Qty) ✓✓

[15]

QUESTION 5

5.1 Model answer: (5)

```
CREATE TABLE Enrolment (
    studentNo   INT,
    moduleCode  VARCHAR(10),
    year        INT,
    mark        INT CHECK (mark BETWEEN 0 AND 100),                    ✓ CHECK
    PRIMARY KEY (studentNo, moduleCode, year),                         ✓ composite PK
    FOREIGN KEY (studentNo) REFERENCES Student(studentNo),             ✓
    FOREIGN KEY (moduleCode) REFERENCES Module(moduleCode)             ✓
);                                                                     ✓ columns with types
```

5.2 INSERT INTO Enrolment (studentNo, moduleCode, year, mark) VALUES (2203, 'COS202', 2026, NULL); ✓✓ (also accept leaving mark out of the column list) (2)

5.3 SELECT name, surname ✓ FROM Student WHERE programme = 'CS' ✓ ORDER BY surname; ✓ (3)

5.4 SELECT moduleCode, AVG(mark) ✓ FROM Enrolment GROUP BY moduleCode ✓ HAVING AVG(mark) > 60; ✓✓ (4)

5.5 SELECT s.name, m.moduleName, e.mark ✓
FROM Enrolment e JOIN Student s ON e.studentNo = s.studentNo ✓
JOIN Module m ON e.moduleCode = m.moduleCode ✓
WHERE e.mark >= 70; ✓ (4)

5.6 COS201 69 ✓ (72 + 45 + 90 = 207, ÷ 3) COS203 81 ✓ INF101 66 ✓ (COS202 is left out: 58) (3)

5.7 UPDATE Enrolment SET mark = mark + 5 ✓ WHERE moduleCode = 'COS202'; ✓ (2)

5.8 DELETE FROM Enrolment ✓ WHERE mark IS NULL; ✓ (= NULL gets no mark) (2)

5.9 SELECT name FROM Student ✓ WHERE studentNo NOT IN ✓ (SELECT studentNo FROM Enrolment WHERE moduleCode = 'COS201'); ✓ Result: Pieter (3)

5.10 WHERE filters individual rows before grouping ✓; HAVING filters groups after GROUP BY and can use aggregate functions such as AVG() ✓ (2)

[30]

QUESTION 6

6.1 Two transactions read the same balance, e.g. R500 ✓. One adds R100 (writes R600), the other subtracts R50 (writes R450) ✓. The second write overwrites the first, so the deposit of R100 is lost; the correct balance is R550 ✓ (3)

6.2 A transaction must get a lock (an exclusive lock for writing) on the data before changing it ✓; the other transaction must wait until the lock is released, so the updates happen one after the other ✓ (2)

6.3 An attack where SQL code is typed into an input field and becomes part of the query, so the attacker can read or change data ✓✓. Prevention, any ONE: use prepared statements (parameterised queries); validate and escape input; give the application account only the rights it needs ✓ (3)

6.4 Data can be lost through hardware failure, errors, theft or ransomware, so a copy is needed to restore it ✓. After a crash the log is used to redo committed transactions and undo uncommitted ones since the last backup or checkpoint ✓ (2)

[10]

TOTAL: 100

END OF MARKING GUIDELINE